

Day 46 - Polynomial Regression

In this notebook, I learned about **Polynomial Regression**, an extension of Linear Regression that models **non-linear relationships** between features and the target variable.

While Linear Regression fits a straight line, Polynomial Regression fits a **curved line** by introducing polynomial terms (squared, cubic, etc.).

Polynomial Regression

Why Polynomial Regression?

- Linear Regression works well only if the data has a **linear trend**.
- If the relationship between X and y is **curved**, Linear Regression fails to capture it.
- Polynomial Regression helps by adding new features:
 - $(X^2, X^3, \dots, X^n)$
- This transforms the linear model into a curve-fitting model.

Polynomial Regression is a type of regression analysis used to model the relationship between a single independent variable (x) and a dependent variable (y) as an **n -th degree polynomial**. Unlike simple linear regression, which fits a straight line to the data, polynomial regression fits a curved line.

This technique is used when the data points show a non-linear pattern, meaning a straight line cannot accurately represent the relationship.

The equation for Simple Linear Regression is:

$$y = b_0 + b_1x$$

The equation for Polynomial Regression is:

$$y = b_0 + b_1x + b_2x^2 + \dots + b_nx^n$$

- x is the independent variable.
- y is the dependent variable.
- $b_0, b_1, \dots, b_n$ are the coefficients (parameters) of the model.
- n is the degree of the polynomial. A higher degree can fit the training data better, but may lead to **overfitting**.

The core idea is to transform the independent variable by adding higher-order terms ($x^2, x^3, \dots$) to the model. We can then use a standard `LinearRegression` model to train on these new features.

Import Libraries

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

Load the dataset

```
In [2]: data = pd.read_csv(r'C:\Users\Arman\Downloads\dataset\emp_sal.csv')
```

```
In [4]: data
```

```
Out[4]:
```

	Position	Level	Salary
0	Jr Software Engineer	1	45000
1	Sr Software Engineer	2	50000
2	Team Lead	3	60000
3	Manager	4	80000
4	Sr manager	5	110000
5	Region Manager	6	150000
6	AVP	7	200000
7	VP	8	300000
8	CTO	9	500000
9	CEO	10	1000000

Separate Features and Target Variable

I have define `X` as the `Level` column (independent variable) and `y` as the `Salary` column (dependent variable).

```
In [5]: X = data.iloc[:, 1:2].values  
y = data.iloc[:,2].values
```

```
In [7]: print(X)
```

```
[[ 1]  
 [ 2]  
 [ 3]  
 [ 4]  
 [ 5]  
 [ 6]  
 [ 7]  
 [ 8]  
 [ 9]  
 [10]]
```

```
In [8]: print(y)
```

```
[ 45000  50000  60000  80000 110000 150000 200000 300000 500000  
1000000]
```

Train a Simple Linear Regression Model (Degree 1)

First, let's train a standard linear model and see how it fits the data. Here I skipped the train-test split for this small dataset to ensure all points are used for training and visualization.

```
In [9]: lin_reg = LinearRegression()  
lin_reg.fit(X,y)
```

```
Out[9]:
```

LinearRegression ⓘ ?

LinearRegression()

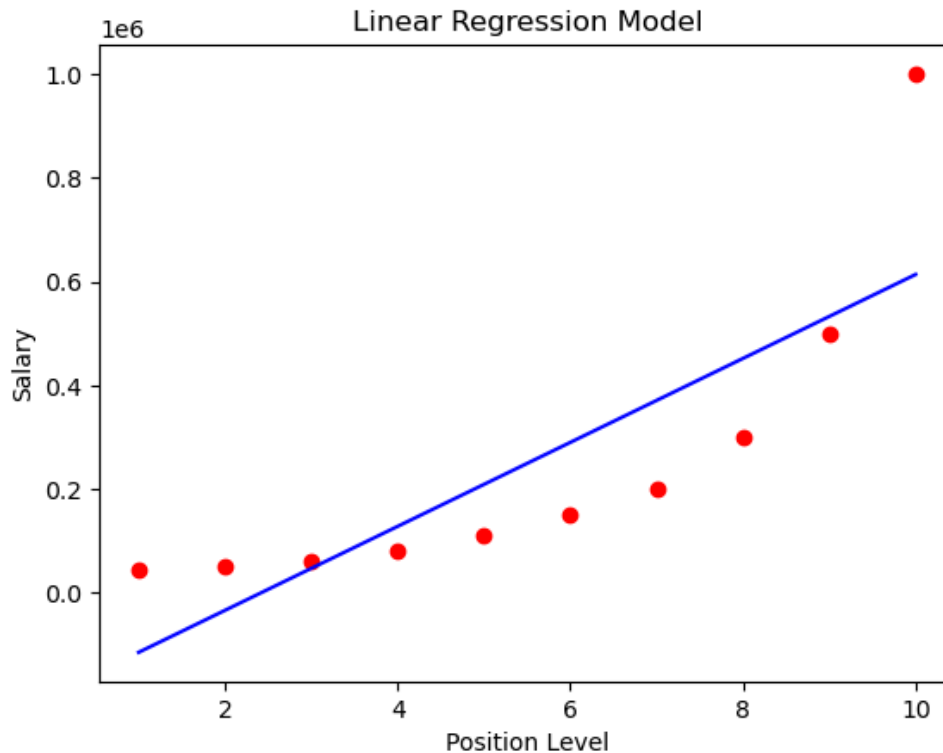
Predict Salary for Level 6 using the linear model

```
In [11]: print(lin_reg.predict([[6]]))
```

[289939.39393939]

Plot Linear Regression Model

```
In [12]: plt.scatter(X,y, color = 'red') #Real data
plt.plot(X, lin_reg.predict(X), color = 'blue') #predicted data
plt.title("Linear Regression Model")
plt.xlabel("Position Level")
plt.ylabel("Salary")
plt.show()
```



Train a Polynomial Regression – Degree 2

This is the core of the notebook. I used `PolynomialFeatures` to create new features that are powers of our original `X` feature. Then, I trained a new `LinearRegression` model on these transformed features.

```
In [17]: poly_reg2 = PolynomialFeatures()
X_poly2 = poly_reg2.fit_transform(X)

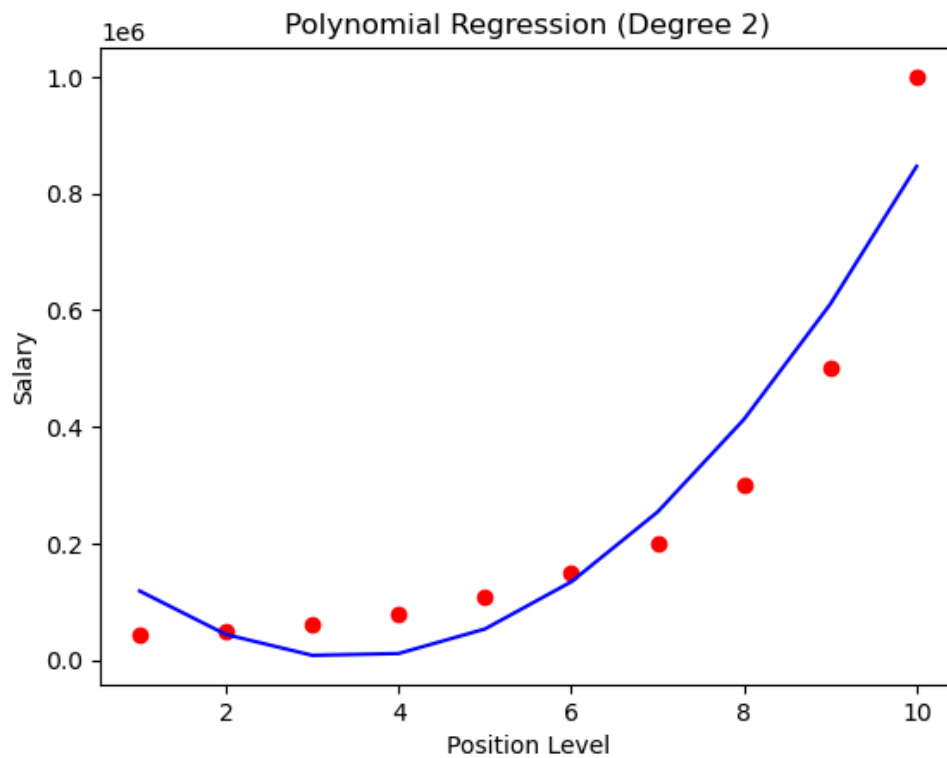
lin_reg2 = LinearRegression()
lin_reg2.fit(X_poly2, y)

print(lin_reg2.predict(poly_reg2.transform([[6]])))
```

[134484.84848485]

Plot Polynomial Regression Model - Degree 2

```
In [15]: plt.scatter(X, y, color='red')
plt.plot(X, lin_reg2.predict(X_poly2), color='blue')
plt.title("Polynomial Regression (Degree 2)")
plt.xlabel("Position Level")
plt.ylabel("Salary")
plt.show()
```



Train a Polynomial Regression – Degree 3

```
In [16]: poly_reg3 = PolynomialFeatures(degree=3)
X_poly3 = poly_reg3.fit_transform(X)

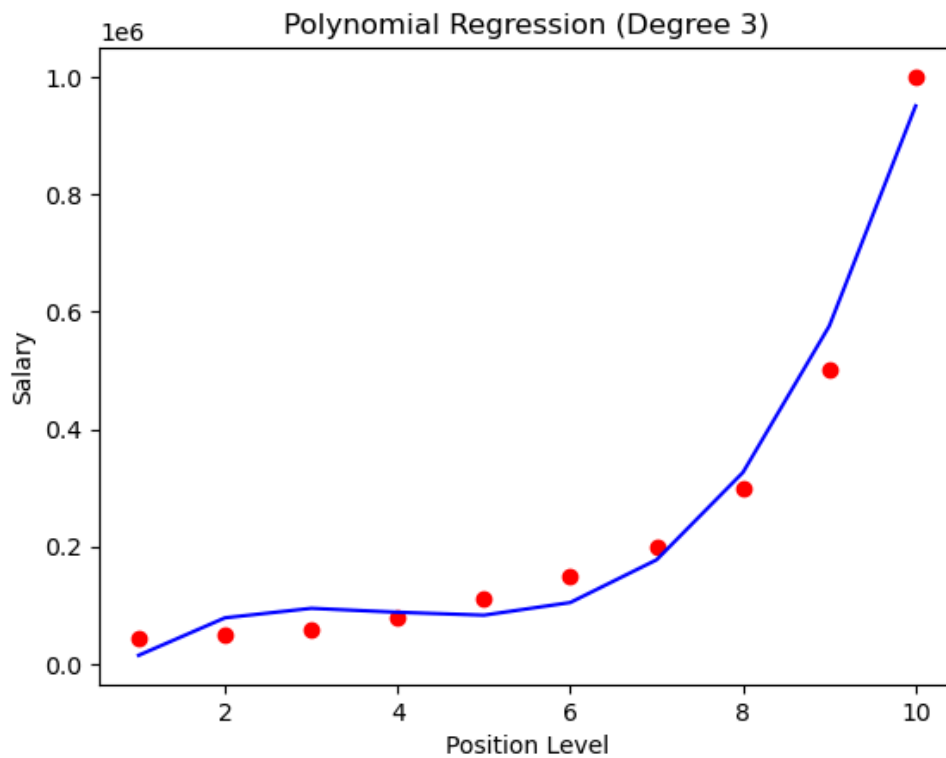
lin_reg3 = LinearRegression()
lin_reg3.fit(X_poly3, y)

print(lin_reg3.predict(poly_reg3.transform([[6]])))
```

[104820.51282051]

Plot Polynomial Regression Model - Degree 3

```
In [18]: plt.scatter(X, y, color='red')
plt.plot(X, lin_reg3.predict(X_poly3), color='blue')
plt.title("Polynomial Regression (Degree 3)")
plt.xlabel("Position Level")
plt.ylabel("Salary")
plt.show()
```



Train a Polynomial Regression – Degree 4

```
In [19]: poly_reg4 = PolynomialFeatures(degree=4)
X_poly4 = poly_reg4.fit_transform(X)

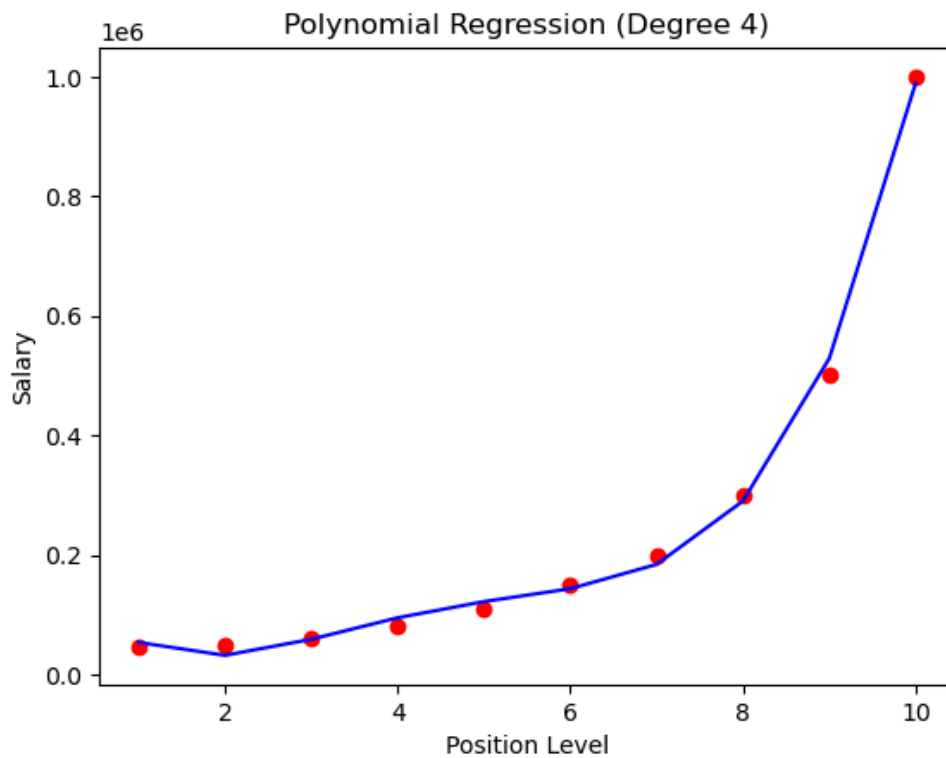
lin_reg4 = LinearRegression()
lin_reg4.fit(X_poly4, y)

print(lin_reg4.predict(poly_reg4.transform([[6]])))
```

[143275.05827508]

Plot Polynomial Regression Model - Degree 4

```
In [20]: plt.scatter(X, y, color='red')
plt.plot(X, lin_reg4.predict(X_poly4), color='blue')
plt.title("Polynomial Regression (Degree 4)")
plt.xlabel("Position Level")
plt.ylabel("Salary")
plt.show()
```



Train a Polynomial Regression – Degree 5

```
In [21]: poly_reg5 = PolynomialFeatures(degree=5)
X_poly5 = poly_reg5.fit_transform(X)

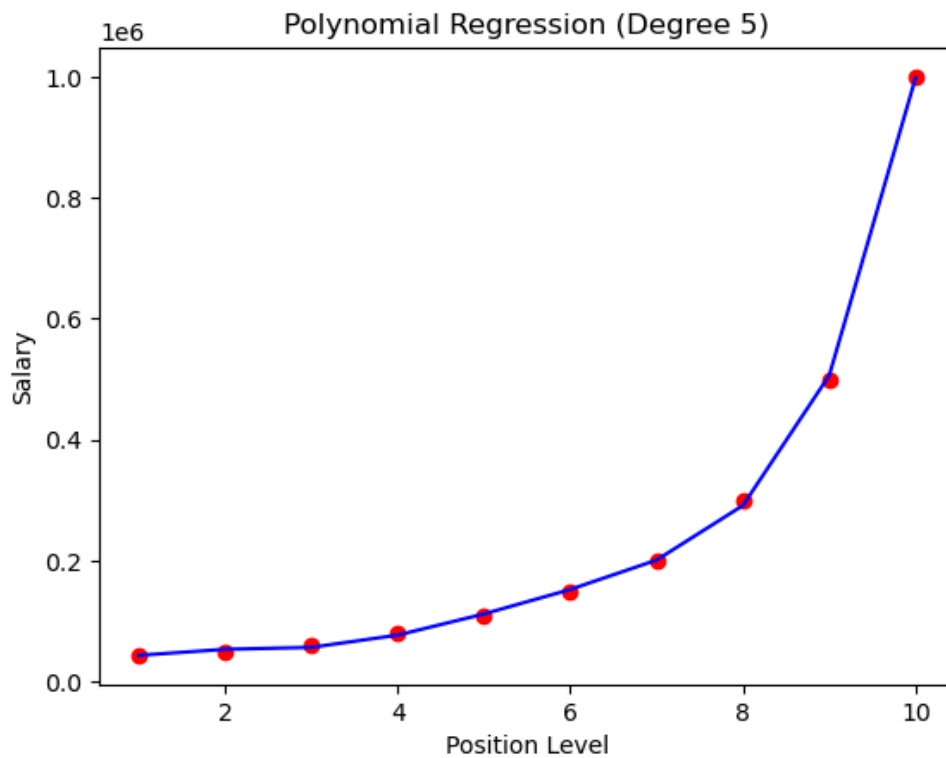
lin_reg5 = LinearRegression()
lin_reg5.fit(X_poly5, y)

print(lin_reg5.predict(poly_reg5.transform([[6]])))
```

[152736.59673623]

Plot Polynomial Regression Model - Degree 5

```
In [22]: plt.scatter(X, y, color='red')
plt.plot(X, lin_reg5.predict(X_poly5), color='blue')
plt.title("Polynomial Regression (Degree 5)")
plt.xlabel("Position Level")
plt.ylabel("Salary")
plt.show()
```



Train a Polynomial Regression – Degree 6

```
In [23]: poly_reg6 = PolynomialFeatures(degree=6)
X_poly6 = poly_reg6.fit_transform(X)

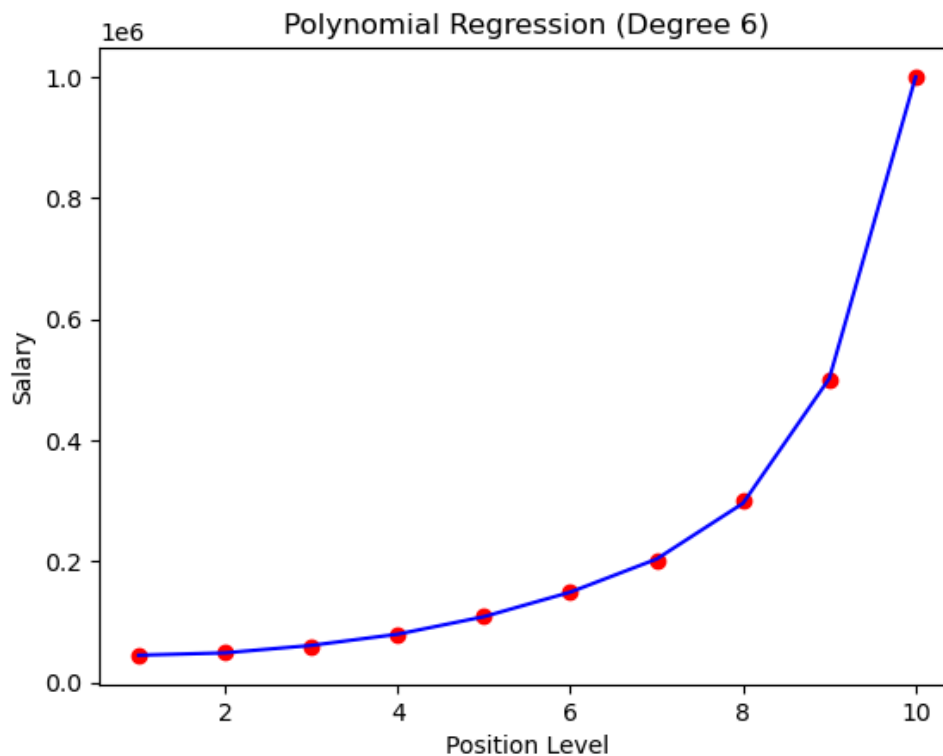
lin_reg6 = LinearRegression()
lin_reg6.fit(X_poly6, y)

print(lin_reg6.predict(poly_reg6.transform([[6]])))
```

[149282.05128119]

Plot Polynomial Regression Model - Degree 6

```
In [24]: plt.scatter(X, y, color='red')
plt.plot(X, lin_reg6.predict(X_poly6), color='blue')
plt.title("Polynomial Regression (Degree 6)")
plt.xlabel("Position Level")
plt.ylabel("Salary")
plt.show()
```



```
In [28]: # Save both poly and model
with open("polynomial_model.pkl", "wb") as f:
    pickle.dump({"poly": poly_reg5, "model": lin_reg5}, f)
```

Conclusion

In this notebook, I learned about **Polynomial Regression** and its importance for modeling non-linear data. By transforming our single feature into a set of polynomial features, I was able to create a much more accurate model that captures the curvature in the dataset. The visualization clearly showed how the polynomial model's curve provides a better fit compared to the straight line of the simple linear model. This technique is a powerful tool for handling complex relationships in your data.

Summary – Day46: Polynomial Regression

- Polynomial Regression is an extension of Linear Regression used to model **non-linear relationships**.
- Instead of fitting just a straight line, it introduces **polynomial terms** ($X^2, X^3, \dots$) to capture curves in the data.
- I compared polynomial degrees **2 to 6**:
 - **Degree 2**: Better than linear, but still a bit simple.
 - **Degree 3–4**: Captures non-linear patterns well, usually the best balance.
 - **Degree 5–6**: Fits training data almost perfectly, but with a risk of **overfitting**.
- A higher degree always looks better on training data, but it may not generalize to unseen data.
- The key takeaway is the importance of choosing the **right polynomial degree** to balance bias and variance.